

Heaven's Light Is Our Guide



Rajshahi University of Engineering & Technology
Department of Electrical & Computer Engineering

Course Title
Digital Techniques Sessional

Course No: ECE 2112

Lab No: 01

Report Title:

Implementation of Logic Gates Using Universal Gates, Full Adder,
and Binary to BCD Converter

Date of Submission: 31 July, 2026

Submitted To	Submitted By
MST. Mazeda Noor Tasnim Lecturer, Department of Electrical & Computer Engineering, RUET	Nujhat Binte Nahira Roll: 2410046 ECE-24 Series

Problem: 01

Problem Definition: Implementation of an OR gate using only NOR gates.

Objective:

To design and verify an OR gate using two NOR gates, and to observe how universal gates can replicate the functionality of other basic gates.

Theory:

A NOR gate is called a universal gate because any other logic gate can be constructed using only NOR gates. Since a NOR gate is itself the complement of an OR gate, i.e., $Y = \overline{A + B}$, feeding this output back into a second NOR gate with both its inputs tied together produces the complement of $\overline{A + B}$, which is simply $A + B$. This works because tying both inputs of a NOR gate to the same signal X forces $Y = \overline{X + X} = \overline{X}$, so the NOR gate behaves as an inverter (NOT gate). Hence:

$$G_1 = \overline{A + B}, \quad Y = \overline{G_1 + G_1} = \overline{G_1} = A + B$$

Truth Table:

A	B	G_1	$Y = A + B$
0	0	1	0
0	1	0	1
1	0	0	1
1	1	0	1

Table 1: Caption

Circuit Diagram:

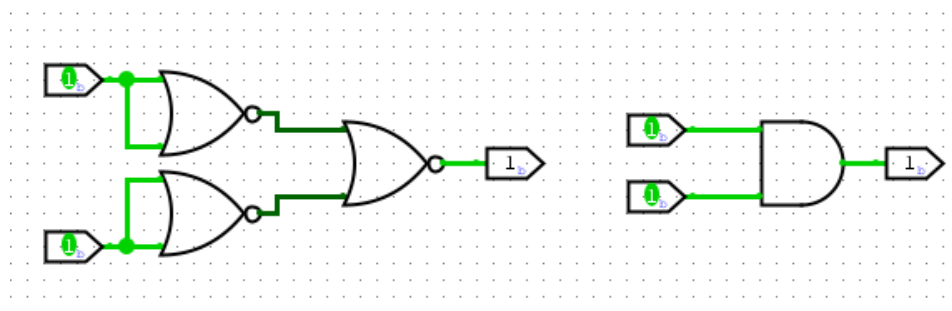


Figure 1: OR using NOR

Discussion:

The output of the first NOR gate, $\overline{A + B}$, is fed into both inputs of the second NOR gate, which forces it to act purely as an inverter. Inverting $\overline{A + B}$ once more restores the original OR relationship, $A + B$. The truth table confirms that the final output Y

matches the standard OR gate output for every combination of A and B . This construction requires exactly two NOR gates, illustrating why NOR is classified as a universal gate.

Conclusion:

The OR gate was successfully implemented using two NOR gates, and the truth table verified that the circuit output matches the expected OR operation for all input combinations.

Problem: 02

Problem Definition: Implementation of an OR gate using only NAND gates.

Objective:

To design and verify an OR gate using three NAND gates, applying De Morgan's theorem.

Theory:

By De Morgan's theorem, $A + B = \overline{\overline{A} \cdot \overline{B}}$. This expression is exactly the definition of a NAND operation performed on $\overline{A}$ and $\overline{B}$. Since a NAND gate with both its inputs tied together behaves as an inverter ($\overline{A \cdot A} = \overline{A}$), the complements of A and B can each be produced using one NAND gate configured this way. A third NAND gate then combines these two complemented signals to produce the final OR output:

$$\overline{A} = \overline{A \cdot A}, \quad \overline{B} = \overline{B \cdot B}, \quad Y = \overline{\overline{A} \cdot \overline{B}} = A + B$$

Truth Table:

A	B	$\overline{A}$	$\overline{B}$	$Y = A + B$
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

Table 2: Caption

Circuit Diagram:

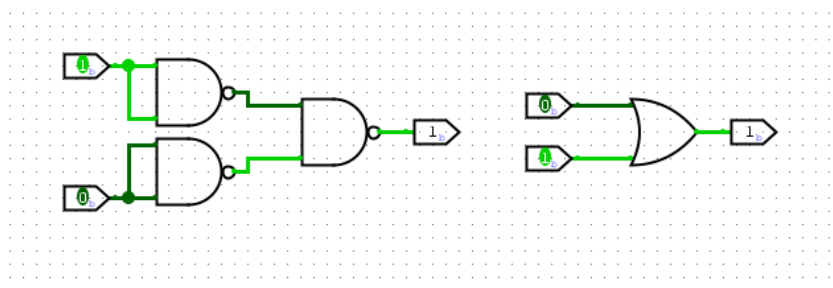


Figure 2: OR using NAND

Discussion:

The first two NAND gates, with their inputs shorted together, act as inverters producing $\overline{A}$ and $\overline{B}$. The third NAND gate then NANDs these two complemented signals together, which by De Morgan's theorem is equivalent to $A + B$. This construction requires three NAND gates, one more than the NOR-based implementation, since NAND is naturally suited to producing AND-like combinations and needs an extra inversion stage to realize OR.

Conclusion:

The OR gate was successfully implemented using three NAND gates, and the resulting truth table matched the standard OR truth table for all four input combinations.

Problem: 03

Problem Definition: Implementation of an AND gate using only NOR gates.

Objective: To design and verify an AND gate using three NOR gates, applying De Morgan's theorem.

Theory:

By De Morgan's theorem, $A \cdot B = \overline{\overline{A} + \overline{B}}$. This is exactly the NOR operation performed on $\overline{A}$ and $\overline{B}$. Since a NOR gate with both inputs tied together acts as an inverter ($\overline{A + A} = \overline{A}$), the complements of A and B are each generated using one NOR gate configured this way, and a third NOR gate combines them to produce the AND output:

$$\overline{A} = \overline{A + A}, \quad \overline{B} = \overline{B + B}, \quad Y = \overline{\overline{A} + \overline{B}} = A \cdot B$$

Truth Table:

A	B	$\overline{A}$	$\overline{B}$	$Y = A \cdot B$
0	0	1	1	0
0	1	1	0	0
1	0	0	1	0
1	1	0	0	1

Table 3: Caption

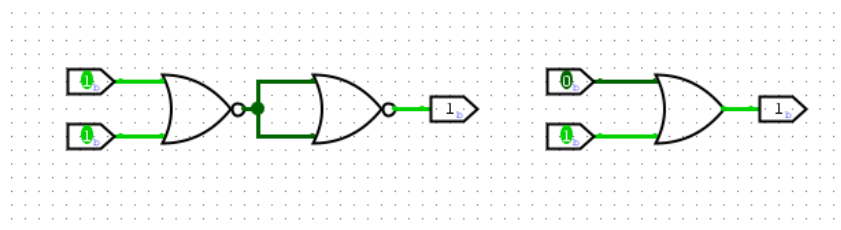
Circuit Diagram:

Figure 3: AND using NOR

Discussion:

The first two NOR gates, with their inputs shorted together, invert A and B to produce $\overline{A}$ and $\overline{B}$. The third NOR gate combines these complements, which by De Morgan's theorem yields the AND function $A \cdot B$. As with the NAND-based OR gate, this implementation requires three gates because NOR naturally produces OR-like combinations, and an additional inversion stage is needed to realize AND.

Conclusion:

The AND gate was successfully implemented using three NOR gates, and the truth table confirmed correct AND behaviour for all input combinations.

Problem: 04

Problem Definition: Implementation of an AND gate using only NAND gates.

Objective:

To design and verify an AND gate using two NAND gates.

Theory:

A NAND gate is the complement of an AND gate, i.e., $Y = \overline{A \cdot B}$. Feeding this output into a second NAND gate with both inputs tied together inverts it once more, restoring the original AND function:

$$G_1 = \overline{A \cdot B}, \quad Y = \overline{G_1 \cdot G_1} = \overline{G_1} = A \cdot B$$

Truth Table:

A	B	$G_1 = \overline{A \cdot B}$	$Y = A \cdot B$
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 4: Caption

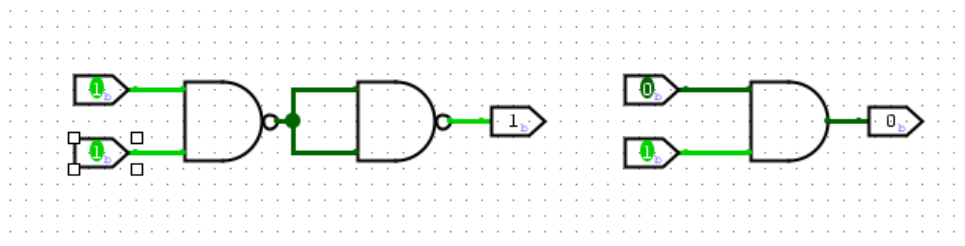
Circuit Diagram:

Figure 4: AND using NAND

Discussion:

The output of the first NAND gate, $\overline{A \cdot B}$, is fed into both inputs of the second NAND gate, forcing it to act as an inverter. This second inversion restores the original AND relationship. Since NAND is classified as a universal gate, only two NAND gates are needed to implement AND, exactly mirroring the NOR-based OR gate implementation in Problem 1.

Conclusion:

The AND gate was successfully implemented using two NAND gates, and the resulting truth table matched the standard AND truth table for all input combinations.

Problem: 05

Problem Definition: Implementation of a NOT gate using only a NOR gate.

Objective:

To design and verify a NOT gate (inverter) using a single NOR gate.

Theory:

The NOR operation is defined as $Y = \overline{A + B}$. If both inputs of the NOR gate are tied together and driven by the same signal A , the expression reduces to $Y = \overline{A + A} = \overline{A}$, which is precisely the NOT operation.

Truth Table:

A	$Y = \overline{A}$
0	1
1	0

Table 5: Caption

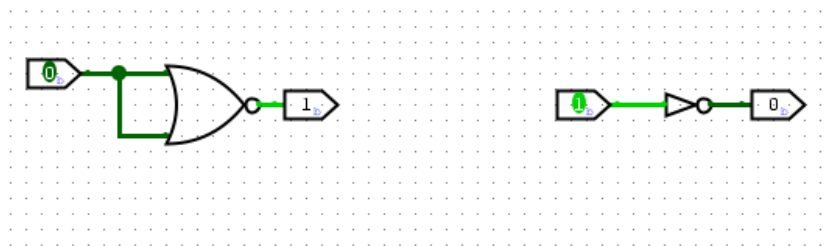
Circuit Diagram:

Figure 5: NOT using NOR

Discussion:

Tying both inputs of the NOR gate to the same signal eliminates the OR-combination behaviour of the gate, leaving only the inversion implicit in the NOR operation. This is the simplest possible universal-gate construction, requiring only a single gate, and forms the basic inverter building block used in Problems 1 and 3 above.

Conclusion:

A NOT gate was successfully implemented using a single NOR gate with its inputs shorted together, and the truth table confirmed correct inversion for both input states.

Problem: 06

Problem Definition: Implementation of a NOT gate using only a NAND gate.

Objective:

To design and verify a NOT gate (inverter) using a single NAND gate.

Theory:

The NAND operation is defined as $Y = \overline{A \cdot B}$. If both inputs of the NAND gate are tied together and driven by the same signal A , the expression reduces to $Y = \overline{A \cdot A} = \overline{A}$, which is precisely the NOT operation.

Truth Table:

A	$Y = \overline{A}$
0	1
1	0

Table 6: Caption

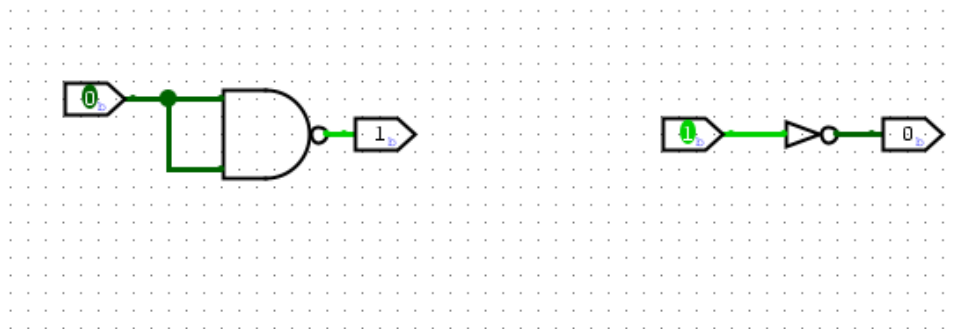
Circuit Diagram:

Figure 6: NOT using NAND

Discussion:

Just as with the NOR-based inverter, shorting both inputs of the NAND gate to the same signal removes the AND-combination behaviour, leaving only the inversion implicit in the NAND operation. This is the basic inverter building block used in Problems 2 and 4 above, and confirms that both NOR and NAND gates are independently sufficient to build every other basic logic gate.

Conclusion:

A NOT gate was successfully implemented using a single NAND gate with its inputs shorted together, and the truth table confirmed correct inversion for both input states.

Problem: 07

Problem Definition: Implementation of a full adder circuit and testing it against its truth table.

Objective:

To design a full adder using basic logic gates and verify its Sum and Carry outputs for all eight possible input combinations of A , B , and C_{in} .

Theory:

A full adder is a combinational circuit that adds three single-bit inputs – two significant bits A and B , and a carry-in bit C_{in} from a previous addition stage – and produces two outputs: a Sum bit and a carry-out bit C_{out} . Unlike a half adder, which only adds two bits and cannot accept an incoming carry, a full adder can be cascaded to build multi-bit adders. The Boolean expressions for the outputs are:

$$Sum = A \oplus B \oplus C_{in}, \quad C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$$

A full adder can be built from two half adders and an OR gate: the first half adder computes $P = A \oplus B$ and $A \cdot B$; the second half adder takes P and C_{in} to compute the final Sum and the product $P \cdot C_{in}$; the OR gate then combines the two partial carry terms $A \cdot B$ and $P \cdot C_{in}$ to produce C_{out} .

Truth Table:

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 7: Caption

Circuit Diagram:

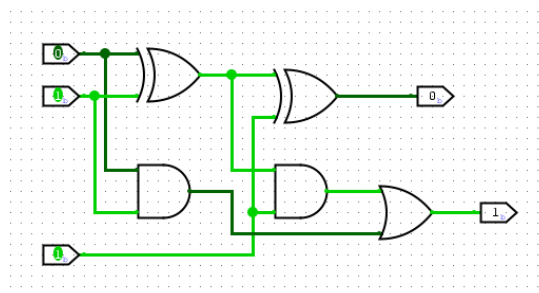


Figure 7: FULL ADDER Circuit

Discussion:

The first XOR gate computes $P = A \oplus B$, the partial sum of A and B without considering any carry-in. The first AND gate simultaneously computes $A \cdot B$, the carry generated directly by A and B . The second XOR gate then combines P with C_{in} to produce the final Sum bit, while the second AND gate computes $P \cdot C_{in}$, the carry generated when C_{in} interacts with P . Finally, the OR gate combines $A \cdot B$ and $P \cdot C_{in}$ to produce C_{out} , since a carry-out can be generated by either condition. Testing the circuit against all eight rows of the truth table confirmed that both Sum and C_{out} matched the expected values in every case, validating the correctness of the design.

Conclusion:

A full adder was successfully implemented using two XOR gates, two AND gates, and one OR gate. Verification against the complete truth table for all eight input combinations confirmed that the circuit correctly computes both the Sum and Carry-out for three-bit binary addition.

Problem: 08

Problem Definition: Implementation of a binary to BCD converter for a 4-bit binary input.

Objective:

To design a combinational logic circuit that converts a 4-bit binary number (representing decimal values 0 to 15) into its equivalent two-digit Binary Coded Decimal (BCD) representation.

Theory:

A 4-bit binary number $B_3B_2B_1B_0$ can represent decimal values from 0 to 15. Since BCD represents each decimal digit using 4 bits and only allows digit values from 0 to 9, any binary value from 10 to 15 must be expressed using two BCD digits: a tens digit T (which can only be 0 or 1 in this range) and a units digit $U_3U_2U_1U_0$ (0 to 9). If the binary value is 9 or less, $T = 0$ and the units digit equals the binary input directly. If the binary value is 10 or more, $T = 1$ and the units digit equals the binary value minus 10.

Tabulating the input-output relationship for all 16 combinations and simplifying using Karnaugh maps yields the following minimized Boolean expressions:

$$T = B_3(B_2 + B_1)$$

$$U_3 = B_3\overline{B_2}\overline{B_1}$$

$$U_2 = B_2\overline{B_3} + B_2B_1$$

$$U_1 = B_1\overline{B_3} + B_2\overline{B_1}B_3$$

$$U_0 = B_0$$

Note that U_0 always equals B_0 directly, since subtracting 10 (which is even) never changes the parity of the least significant bit.

Truth Table:

B_3	B_2	B_1	B_0	T	U_3	U_2	U_1	U_0
0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	1
0	0	1	0	0	0	0	1	0
0	0	1	1	0	0	0	1	1
0	1	0	0	0	0	1	0	0
0	1	0	1	0	0	1	0	1
0	1	1	0	0	0	1	1	0
0	1	1	1	0	0	1	1	1
1	0	0	0	0	1	0	0	0
1	0	0	1	0	1	0	0	1
1	0	1	0	1	0	0	0	0
1	0	1	1	1	0	0	0	1
1	1	0	0	1	0	0	1	0
1	1	0	1	1	0	0	1	1
1	1	1	0	1	0	1	0	0
1	1	1	1	1	0	1	0	1

Table 8: Caption

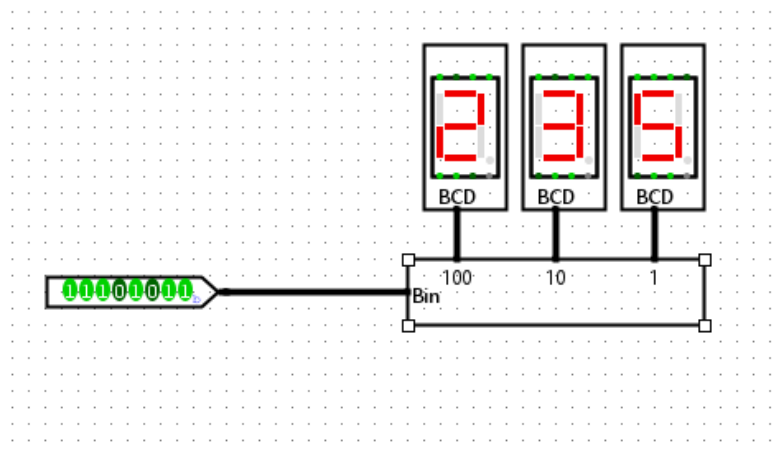
Circuit Diagram:

Figure 8: Binary to BCD

Discussion:

Each output of the converter is an independent Boolean function of the four input bits $B_3B_2B_1B_0$, obtained directly from Karnaugh map simplification of the 16-row truth table. The tens bit T behaves as a simple threshold detector, going high only when the input reaches 10 or above. The units bits $U_3U_2U_1U_0$ effectively subtract 10 from the input whenever $T = 1$, which is why several of their expressions share terms with the expression for T . This block can be realized with AND, OR, and NOT gates directly from the minimized equations, without needing a separate adder-subtractor stage, since the input range is restricted to 4 bits (0-15).

Conclusion:

A binary to BCD converter for 4-bit binary inputs (0 to 15) was successfully designed using combinational logic derived from Karnaugh map simplification. The truth table confirmed that the circuit correctly separates each binary input into its tens bit T and units nibble $U_3U_2U_1U_0$, matching the expected two-digit BCD representation for every input value.

References

1. M. M. Mano, *Digital Design*, 5th ed. Upper Saddle River, N.J.: Pearson, 2013.
2. R. J. Tocci, N. S. Widmer, and G. L. Moss, *Digital Systems: Principles and Applications*, 11th ed. Upper Saddle River, N.J.: Pearson, 2011.